



ADS04：左式堆与斜堆

来源：Materials/Student Slides/ADS04LeftistSkewH_Stu.ppt

课件范围：S1-S4、S5-S7、S8-S10、S11-S13；共 13 张幻灯片。

笔记按原课件顺序整理；S 编号为幻灯片顺序。

本讲路线图

- 普通堆的合并问题 → 左式堆的右路径约束 → 合并与删最小元 → 斜堆的无状态交换 → 用势能法证明摊还界。

1. 左式堆：把合并集中到短右路径 [S1-S4]

左式堆（Leftist Heap）仍满足最小堆的次序条件，但不要求普通二叉堆的完全树形状。它通过约束每个节点的左右子树，使合并只需沿一条短的右路径进行。[S1-S2]

1.1 目标与两类性质 [S2]

重要性：A · 必须掌握

1. **次序性质（Heap Order）**。每个节点的键不大于其孩子的键，因此根存放最小元；这一点与普通最小堆相同。
2. **结构性性质（Structure Property）**。普通二叉堆用完全二叉树保持形状；左式堆仅要求是一棵满足下节偏向条件的二叉树，允许明显不平衡。
3. **操作目标**。若把两个普通二叉堆的元素放在一起再建堆，合并可耗费 $O(N)$ ；左式堆希望让合并沿较短的右路径完成。[S2]

解释补充：这里的 N 指合并涉及的总节点数；讨论不同操作的复杂度时，还需要明确是单次最坏界还是多次操作的摊还界。

1.2 空路径长与左式条件 [S3]

重要性：A · 必须掌握

空路径长（Null Path Length, NPL） $npl(x)$ 是从节点 x 到第一个“不同时拥有两个孩子”的节点的最短路径长度，并约定 $npl(\text{NULL}) = -1$ 。因此无孩子或只有一个孩子的节点，其 NPL 为 0。[S3]

按节点情况分类：

1. **空指针：** $npl(\text{NULL}) = -1$ 。
2. **至少有一个空孩子：**最短路径立即到达不具备两个孩子的节点，故 $npl(x) = 0$ 。
3. **两个孩子都存在：** $npl(x) = 1 + \min\{npl(x_L), npl(x_R)\}$ 。

左式条件：每个节点都满足 $npl(x_L) \geq npl(x_R)$ 。于是上式化为 $npl(x) = npl(x_R) + 1$ ：沿右孩子前进时，NPL 每步减 1。这保证右路径短，却允许左子树很深。[S3]

1.3 右路径长度的对数界 [S4]

重要性：A · 必须掌握

定理：若一棵左式树的右路径有 r 个节点，则整棵树至少有 $2^r - 1$ 个节点。因此 N 节点左式树的右路径长度满足

$$r \leq \log_2(N + 1) = O(\log N).$$

核心原理

左式条件使右路径上的 NPL 逐级减一；要达到给定 NPL，左右两个子树都必须至少具有相应的规模。

1. 结论与记号。设 $m(k)$ 为 NPL 为 k 的左式树可能具有的最少节点数。右路径含 r 个节点等价于根的 NPL 为 $r - 1$ 。
2. 基础情形。 $k = 0$ 时，根本身至少一个节点，所以 $m(0) = 1 = 2^{0+1} - 1$ 。
3. 归纳步骤。若根的 NPL 为 $k \geq 1$ ，右孩子的 NPL 为 $k - 1$ ；左式条件保证左孩子 NPL 至少也是 $k - 1$ 。两个子树各至少有 $2^k - 1$ 个节点，因此 $m(k) \geq 1 + 2(2^k - 1) = 2^{k+1} - 1$ 。
4. 代回右路径。令 $k = r - 1$ ，得 $N \geq 2^r - 1$ ；对两边取以 2 为底的对数，得到 $r \leq \log_2(N + 1)$ 。[S4]

补充推导：S4 给出定理并把证明指向教材第 162 页，上述归纳步骤补全了“左右子树都至少达到 $k - 1$ ”到节点数下界的过渡。提取文字中的 $2r-1$ 丢失了上标；原图明确是 $2^r - 1$ 。

插入可视为与单节点堆合并。由于合并主要影响右路径，对数右路径是左式堆操作快的核心原因。[S4]

2. 左式堆的合并、插入与删最小元 [S5-S7]

左式堆更新的操作是合并（Merge）：维护较小根为新根，递归合并它的右子树，最后恢复左式条件。[S5-S6]

2.1 递归合并的分类步骤 [S5-S6]

重要性：A · 必须掌握

节点通常保存键、左右孩子指针和 NPL。对两个最小左式堆 H_1, H_2 ，合并按以下情况处理：[S5-S6]

1. 空堆边界。若一个堆为空，直接返回另一个堆。
2. 确定新根。比较两堆根键，把较小者记为 H_1 ；新堆根必须是它，才能保持最小堆次序。
3. 递归合并右侧。计算 $H_1.\text{Right} \leftarrow \text{Merge}(H_1.\text{Right}, H_2)$ 。左子树暂时不动，因为可能改变左式条件的是新的右子树。
4. 按 NPL 分类调整。若 $npl(H_1.\text{Left}) < npl(H_1.\text{Right})$ ，交换左右孩子；否则保持原位。
5. 更新状态。令 $npl(H_1) = npl(H_1.\text{Right}) + 1$ ，再返回 H_1 。当原左孩子为空时，课件代码直接把 H_2 接到左边，当前节点 NPL 仍为 0。[S6]

核心原理

每层只沿“较小根的右孩子”递归；回溯时交换左右孩子并更新 NPL，既维持堆序，也恢复右路径较短的结构条件。

递归时， H_1 根键不大于 H_2 根键，也不大于 H_1 原有右子树的键；合并后的右子树仍不可能含有比根更小

的键。交换两个完整子树不会改变根与孩子的大小关系。回溯更新 NPL 后，每层再次满足 $npl(\text{Left}) \geq npl(\text{Right})$ 。漏更新 NPL 会使之后的交换判断依据过期，左式条件可能失效。[S6]

两个输入堆各只沿右路径访问节点，故单次合并的时间为 $O(\log N_1 + \log N_2) = O(\log(N_1 + N_2))$ ；S6 记为 $T_p = O(\log N)$ 。[S4–S6]

2.2 迭代合并、插入和删最小元 [S7]

重要性：B · 重点理解

按操作分类：

1. **迭代合并**。先按键大小合并两个堆的右路径，保留沿途节点原有的左子树；然后自底向上检查左右 NPL，必要时交换孩子，并更新 NPL。[S7]
2. **插入键 x** 。构造只含 x 的单节点左式堆，再与原堆合并；因此插入是合并的特例。[S4、S7]
3. **删除最小元 (DeleteMin)**。删除根后，原根的左右子树本身仍是左式堆；把这两个子树合并，所得根即新的最小元。[S7]

迭代和递归写法遵守同一结构约束。插入与删最小元均以合并为核心，单次时间为 $O(\log N)$ ；读取最小元只看根，为 $O(1)$ 。[S6–S7]

3. 斜堆：省去 NPL 的合并 [S8–S10]

斜堆 (Skew Heap) 也用递归合并维护最小堆次序，但不保存 NPL，不检查左右子树谁更短。它每次局部合并后直接交换左右孩子。[S8]

3.1 合并规则与单次代价 [S8]

重要性：A · 必须掌握

按递归位置分类：

1. **进入递归时**。一个堆为空就返回另一个；否则比较根键，把较小根作为当前根。
2. **递归主体**。合并当前根原来的右子树与另一堆。
3. **回溯时**。无条件交换当前根的左右孩子；无需读写 NPL，也无需测试两侧 NPL 大小。
4. **递归终点**。课件说明：两条原右路径上键值最大的那个末端节点不必再交换孩子；它是合并的自然终点。[S8]

斜堆的右路径**不保证**像左式堆那样逐次保持对数长度；因此某次合并可能很慢。课件的目标是任意连续 M 次操作的总时间 $O(M \log N)$ ，即每次摊还 $O(\log N)$ ，而非保证每次最坏时间都为 $O(\log N)$ 。[S8]

3.2 插入示例与实现取舍 [S9–S10]

重要性：B · 重点理解

S9 用“插入 15”示范把单节点堆与原斜堆合并；同页还把递归合并改写成先组织右路径、再交换孩子的迭代视角。关键是合并过程中根仍取较小键，交换仅改变树形，不改变最小堆次序。[S9]

与左式堆比较：

1. **附加状态**：左式堆每个节点存 NPL；斜堆不存。[S6、S10]
2. **交换条件**：左式堆按 NPL 比较后决定；斜堆在非终点的合并层直接交换。[S6、S8、S10]

3. **复杂度含义：**左式堆右路径始终为 $O(\log N)$ ；斜堆用操作序列的摊还分析给出 $O(\log N)$ ，不提供同样的单次路径保证。[S4、S8、S11–S12]

S10 把两种堆“期望右路径的精确长度”列作课件讨论中的开放问题；这不影响下文对斜堆摊还时间的证明。[S10]

4. 斜堆的势能分析 [S11–S12]

用势能法（Potential Method）给每个堆状态分配势能，以抵消少数昂贵合并的实际代价。插入与删最小元都可化为合并，因此先分析合并。[S11]

4.1 重节点、轻节点与势能 [S11]

重要性：A · 必须掌握

设 $s(p)$ 为以 p 为根的子树节点数，**包含 p 自己**。按右子树规模分类：[S11]

1. **重节点（Heavy Node）：** $s(p_{\text{right}}) \geq s(p)/2$ 。
2. **轻节点（Light Node）：** $s(p_{\text{right}}) < s(p)/2$ 。
3. **势能：**对堆状态 D 定义 $\Phi(D)$ = 该堆中的重节点总数；势能始终非负。

沿右路径每遇到一个轻节点，进入其右子树时规模就少于原来一半。因此含 N 个节点的堆的右路径上，轻节点数 $l = O(\log N)$ ；重节点数 h 单次可以很多。[S11–S12]

4.2 一次合并的摊还界 [S12]

重要性：A · 必须掌握

核心原理

把实际访问的右路径节点分成轻、重两类；旧路径上的重节点带来的大部分实际工作由势能下降支付，剩下的只与对数个轻节点有关。

设输入堆 H_i 的原右路径有 l_i 个轻节点、 h_i 个重节点， $i = 1, 2$ 。两条路径外原有的重节点数记为 h 。合并后只有原右路径上的节点可能改变轻重状态。[S12]

1. **实际代价。**每个原右路径节点至多经过常数次处理，因此 $T_{\text{actual}} = O(l_1 + h_1 + l_2 + h_2)$ ；忽略常数后的课件记法为 $T_{\text{worst}} = l_1 + h_1 + l_2 + h_2$ 。
2. **合并前势能。** $\Phi_i = h_1 + h_2 + h$ 。
3. **合并后势能上界。**原路径外的贡献仍为 h ；原路径上新形成的重节点至多按轻节点数计，故 $\Phi_{i+1} \leq l_1 + l_2 + h$ 。[S12]
4. **计算摊还代价。**在课件的单位代价记法下，

$$\hat{T} = T_{\text{worst}} + \Phi_{i+1} - \Phi_i \leq 2(l_1 + l_2) = O(\log N).$$

5. **推广到序列。**对连续 M 次合并、插入或删最小元求和，势能差首尾相消；由于势能非负，得到总时间 $O(M \log N)$ 的摊还界。[S8、S11–S12]

解释补充：第 3 步的关键不等式由 S12 的图示与势能列式给出。更细的证明要逐个追踪原右路径节点在交换前后的轻重状态；本文保留该界及它在摊还式中的作用，不把单次最坏时间误写成对数级。

5. 阅读材料 [S13]

重要性：C · 了解即可

课件列出 *Data Structures and Algorithm Analysis in C*, 2nd ed., Chapter 5 (pp. 161–169) 与 Chapter 11 (pp. 435–437) 供进一步阅读。[S13]

本讲总结

- 左式堆保留最小堆次序，用 $npl(\text{Left}) \geq npl(\text{Right})$ 限制右路径。[S2–S3]
- 右路径含 r 个节点的左式树至少有 $2^r - 1$ 个节点，所以右路径为 $O(\log N)$ 。[S4]
- 左式堆的合并沿右路径递归，回溯时交换孩子并更新 NPL；插入和删最小元都化为合并。[S5–S7]
- 斜堆省去 NPL 与条件测试，合并时直接交换孩子；单次代价与摊还代价必须区分。[S8–S10]
- 斜堆用“重节点数”作势能，得到每次合并摊还 $O(\log N)$ ，连续 M 次操作总计 $O(M \log N)$ 。[S11–S12]

复杂度与适用条件速查

结构 / 操作	单次最坏时间	摊还时间	条件	来源
左式堆读最小元	$O(1)$	$O(1)$	最小元在根	S2、S7
左式堆合并	$O(\log N)$	$O(\log N)$	每节点保持左式条件	S4–S6
左式堆插入	$O(\log N)$	$O(\log N)$	与单节点堆合并	S4、S7
左式堆删最小元	$O(\log N)$	$O(\log N)$	合并根的两子堆	S7
斜堆合并、插入、删最小元	可达 $O(N)$	$O(\log N)$	不存 NPL；按势能分析序列	S8–S12

易错点

1. NPL 是最短路径长。 $npl(\text{NULL}) = -1$ ，只有一个孩子的节点 NPL 为 0；它不是普通树高。[S3]
2. 左式条件比较子树 NPL。左子树 NPL 不小于右子树，保证短的是右路径，不保证整棵树高度为 $O(\log N)$ 。[S3–S4]
3. 合并后必须更新 NPL。只交换孩子而沿用旧 NPL，会让以后判断用到错误状态。[S6]
4. 斜堆的对数界是摊还界。个别操作沿长右路径可耗时很多，但连续操作的平均代价受势能限制。[S8、S11–S12]
5. 重节点按右子树规模定义。比较的是 $s(p_{\text{right}})$ 与 $s(p)/2$ ； $s(p)$ 包括节点 p 自身。[S11]

复习自测

1. 为什么左式堆只要求短右路径就能加快合并？（来源：[S2–S4]）
2. 对空节点、单孩子节点、双孩子节点分别写出 NPL 的取值规则。（来源：[S3]）
3. 补全 $N \geq 2^r - 1$ 的归纳证明，并推出右路径的长度界。（来源：[S4]）
4. 合并两个左式堆时，怎样分类处理空堆、较小根、左右交换和 NPL 更新？（来源：[S5–S6]）
5. 左式堆的插入、删最小元如何转化为合并？（来源：[S4、S7]）

6. 斜堆为什么不需要保存 NPL? 这与单次最坏时间有什么关系? (来源: [S8–S10])
7. 在斜堆摊还证明中, 为什么右路径上的轻节点只有 $O(\log N)$ 个? 势能差如何抵消重节点的处理代价? (来源: [S11–S12])

术语速查

术语	概念	来源
左式堆 (Leftist Heap)	保持堆序并使右路径较短的可合并堆	S2–S3
空路径长 (Null Path Length, NPL)	到不具有两个孩子的节点的最短路径长	S3
左式条件 (Leftist Property)	$npl(x_L) \geq npl(x_R)$	S3
右路径 (Right Path)	从根反复取右孩子形成的路径	S4
合并 (Merge)	两个堆递归或迭代组成一个堆	S5–S8
斜堆 (Skew Heap)	合并后直接交换孩子、不存 NPL 的堆	S8
重节点 (Heavy Node)	右子树规模至少占当前子树一半	S11
轻节点 (Light Node)	右子树规模不足当前子树一半	S11
势能法 (Potential Method)	以势能变化修正实际代价的摊还分析	S11–S12

待核对

- [待核对 S9] “插入 15” 和迭代合并主要由树形图展示; 文字可确认它们使用合并规则, 但若逐步复刻每次孩子交换, 应对照原图。
- [待核对 S12] 课件给出 $\Phi_{i+1} \leq l_1 + l_2 + h$, 未逐节点展开轻重状态变化的严格证明; 若课程要求形式化证明, 可参考 S13 指定教材相应章节。